

SD Basic - Locks and Transactions

Keeping two sessions out of each other's way, and making several writes all-or-nothing.

Everything on this page exists because more than one session can be running at once. A single-user program never needs any of it; the moment a second person runs the same program, all of it matters.

SD Basic - File Handling introduces `readu` and the `locked` clause. This page is the whole picture: what each lock code means, what the other session sees, what waits and for how long, and what a transaction does to all of it.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The three kinds of lock

record lock	one record of one file. <code>readu</code> , <code>readl</code> , <code>recordlocku</code> , <code>recordlockl</code>
file lock	every record of one file, including ones that do not exist yet. <code>filelock</code>
task lock	one of 64 numbered semaphores that mean whatever you decide. <code>lock</code> , <code>unlock</code>

All three belong to the **session**, not to the program, and all three are released when the session ends.

Record locks

```
readu variable from file.variable, record.id {locked ...} then ... else ...
readl variable from file.variable, record.id {locked ...} then ... else ...
recordlocku file.variable, record.id {locked ...}
recordlockl file.variable, record.id {locked ...}
release {file.variable {, record.id}}
```

`readu` takes an **update** lock, `readl` a **shared read** lock. `recordlocku` and `recordlockl` take the same two locks without reading the record — useful when you are about to create one.

`release` with a file variable and a record id gives back one lock, with just a file variable every lock on that file, and with **no arguments at all** every lock the session holds.

A `write` **outside a transaction releases the lock; inside one it keeps it.** On the same record in the same program:

`RECORDLOCKED()` AFTER

<code>readu</code> then <code>write</code> , no transaction	0 — the lock is gone
<code>readu</code> then <code>write</code> , inside a transaction	2 — still held

That is the single most useful fact about the two. Outside a transaction the read-modify-write cycle ends when you write. Inside one, it ends when the transaction does.

RECORDLOCKED() — what every return value means

```
recordlocked(file.variable, record.id)
```

Every one of the seven codes below was produced deliberately, three of them by a second session holding the lock:

CODE	MEANING
-3	another session holds a file lock on this file
-2	another session holds an update lock on this record
-1	another session holds a read lock on this record
0	nobody holds a lock on it
1	this session holds a read lock
2	this session holds an update lock
3	this session holds a file lock

And `status()` **after it is the other session's user number.** This is not written down anywhere else and it is the only way to find out *who* is holding the record. With the holder running as user 73 and the asker as user 74:

```
if recordlocked(f, id) < 0 then
  crt 'Record ' : id : ' is held by user ' : status()
end
```

`status()` read **73** after `recordlocked()`, after a refused `readu`, after a refused `readl`, after a refused `filelock` and after a refused `lock` — the same number every time. A positive code is your own lock and needs no enquiry.

A negative code is a snapshot, not a promise. The other session may release between your test and your read.

`recordlocked()` is for reporting; the `locked` clause is for deciding.

The `locked` clause, and what happens without one

```
readu rec from f, id locked
  crt 'Someone else is editing that record.'
  return
end then
...
end else
...
end
```

Without a `locked` clause, a conflicting read waits — and it really does wait. Across two sessions, the second session's plain `readu` against a record the first is holding **blocks for 252 ms** and returns the instant the holder releases it, with `recordlocked()` then reading `2`. It does not fail, it does not time out, and there is no message. SD retries every 250 milliseconds, which is where that figure comes from.

A program without a `locked` clause is a program that can sit silently for as long as somebody else keeps a record open. **Any interactive read-for-update should have one.**

With one session holding an update lock on `R1`:

THE SECOND SESSION TRIED	RESULT
<code>readu ... 'R1' locked</code>	the <code>locked</code> branch, <code>status()</code> 73
<code>readl ... 'R1' locked</code>	the <code>locked</code> branch, <code>status()</code> 73
<code>readu ... 'R2' locked</code>	succeeded — a different record is unaffected
plain <code>readu ... 'R1'</code>	waited 252 ms, then got the lock

A read lock is not a free pass. `readl` is refused against another session's update lock, and `readu` is refused against another session's read lock. Two `readl` locks on the same record **do** coexist: both sessions hold one at once and each sees its own as `1`. When the second releases its own, `recordlocked()` goes back to **-1** — the first session's lock is still there. The code reports the lock that matters to you, not a count.

File locks

```
filelock file.variable {locked ...} {on error ...}
fileunlock file.variable
```

`filelock` has no `then` or `else` clause, so success is invisible. It takes only `locked` and `on error`. If you need to know whether you got it, set a flag in the `locked` branch, or ask `recordlocked()` — a `3` means the file lock is yours:

```
blocked = @false
filelock f locked
  blocked = @true
end
if not(blocked) then
  ...
  fileunlock f
end
```

Forgetting the `fileunlock` deadlocks other sessions, including the one that is waiting to tell you it has finished. A rendezvous record written into a file you have locked is a rendezvous that never happens.

With one session holding a file lock:

<code>recordlocked(f, 'R1')</code> from the other session	<code>-3</code> , <code>status()</code> the owner
<code>recordlocked(f, 'R2')</code>	<code>-3</code> as well — every record, not the one you locked
<code>readu ... locked</code> on any record	the <code>locked</code> branch
a plain <code>read</code>	succeeded — <code>[beta]</code> came back normally
one record already locked by somebody else	<code>filelock</code> took the <code>locked</code> branch

A file lock stops locking, not reading. That surprises people. Other sessions can still read every record in the file while you hold it; what they cannot do is take a lock, and therefore cannot write.

Task locks

```
lock n then ... else ...
```

```
unlock n
```

There are **64** of them, numbered 0 to 63, and SD attaches no meaning to any of them. They are for serialising something that is not a record — a nightly report, a printer, an external file.

A `lock` with no `else` clause retries for ever. The compiler generates one for you, and what it generates is `sleep 1` followed by a jump back to the `lock` — so a bare `lock 7` in a program whose lock somebody else holds is an infinite loop with a one second period and no output. Write the `else`.

With one session holding task lock 7:

<code>lock 7</code> in the other session	the <code>else</code> branch, <code>status()</code> 73 — the holder's user number
--	--

<code>lock 8</code>	the <code>then</code> branch, <code>status()</code> 0
---------------------	---

So the `else` branch tells you who has it, exactly as the record lock enquiry does.

Taking a lock you already hold succeeds, and a task lock is not a counter — one `unlock` releases it however many times you locked it. The ownership test is *"unowned or mine"*, and `unlock` clears the slot outright.

`testlock()` would answer *"who owns lock n"* without taking it, and **it is not available to an ordinary program** — see "What is not here" below.

Transactions

```
begin transaction
...
commit
...
rollback
end transaction
```

or, the same thing in the other spelling:

```
transaction start
...
transaction commit
transaction abort
```

Everything written between the start and the `commit` lands together or not at all.

Inside a transaction you must already hold the lock on every record you write or delete. This is the rule that catches everybody, and the error message names the wrong thing entirely:

```
Error 3023 (o/s 0) writing record (Possible full disk?)
```

The disk is not full. 3023 is "attempt to write/delete record with no lock". The same `write` that succeeds outside a transaction fails with 3023 inside one, and succeeds inside one if a `recordlocku` comes first. Outside a transaction no lock is needed, which is why the same line works in testing and fails in production the first time somebody wraps it.

So the shape of a transaction is:

```
begin transaction
  recordlocku f, id
  write rec to f, id
  commit
end transaction
```

`on error` catches it if you would rather test than abort:

```
write rec to f, id on error
  crt 'write refused, status ' : status()  ;* 3023 = no lock held
end
```

WHAT EACH ENDING DOES

One transaction per row, each on its own record:

	THE WRITE	LOCKS	SYSTEM(1008)
<code>commit</code> then <code>end transaction</code>	lands	released	+1
<code>rollback</code>	discarded	released	0
<code>end transaction</code> with neither	discarded, silently	released	0
<code>transaction commit</code>	lands	released	+1
<code>transaction abort</code>	discarded	released	0

Falling out of the bottom of a transaction throws the work away. There is no implicit commit. A `return`, a `goto` or simply reaching `end transaction` without having executed a `commit` discards every write since the start, and nothing is printed. The record still reads whatever it held before.

Inside the transaction your own reads see your own uncommitted writes — a program reads back its own new value before the commit, and a rolled-back `delete` makes the record read `else` inside the transaction and reappear afterwards.

SYSTEM(1007) AND SYSTEM(1008)

<code>system(1007)</code>	the transaction number, or 0 when not in a transaction
---------------------------	---

<code>system(1008)</code>	the transaction level
---------------------------	-----------------------

`system(1007)` is reliable: it is a fresh number for each transaction and reads 0 the moment one ends, either way.

`system(1008)` **is not, and it is a defect.** The level is incremented when a transaction starts and decremented only on the paths that end in a rollback, so **every committed transaction leaves the count one too high for the rest of the session.** In a run of four unnested transactions the first reports level 1 and the fourth reports level 2. **Do not test `system(1008)` to find out whether you are in a transaction — test `system(1007)`.** This is upstream's, not this port's: `sdb64` carries the identical code.

DO NOT NEST TRANSACTIONS

A `commit` inside a Nested transaction abandons the outer one, and the outer write is lost. With an outer transaction writing `R2` and an inner one writing `R3`:

the inner record <code>R3</code>	<code>inner</code> — landed
----------------------------------	-----------------------------

the outer record <code>R2</code>	<code>base</code> — the write vanished
----------------------------------	--

<code>system(1007)</code> after the inner <code>commit</code>	0 — the session is no longer in any transaction
---	--

Nothing is reported. The compiler generates `commit` as a jump past the `end transaction`, so the bookkeeping that would restore the outer transaction never runs, and the outer `commit` then commits an empty cache. **Keep transactions flat**, and note that this includes a subroutine that starts its own transaction while the caller is in one.

`set.trigger` attaches a program that runs on every write or delete. A trigger runs inside the caller's transaction, so it is bound by the same locking rule.

What is not here

`testlock()` and `getlocks()` are internal-only and an ordinary program cannot call them. They are in the compiler's internal intrinsic list, which only a program compiled with `$internal` in an administrator's `SDSYS` session may reach.

And the compiler does not say so. `v = testlock(5)` in an ordinary account produces, at the **last line of the program**:

```
41: Matrix TESTLOCK is not referenced in a DIM statement
WARNING: TESTLOCK is not assigned a value
```

An unknown function name is read as a matrix reference, so the complaint is about a `dim` statement you never wrote, reported nowhere near the line that caused it. If a function you are sure exists produces that message, it exists and you are not allowed to call it.

The same applies to `getlocks()`, which would list every lock in the system. **From an ordinary program the only enquiry available is `recordlocked()`**, and it answers about one record at a time.

`release.lock` is a restricted statement for the same reason and gives "*Unrecognised statement*".

Deadlock detection exists in the C, and takes the `locked` branch when it fires. It was not provoked: two sessions each waiting on the other's record is a state this set has not deliberately created.

See also

SD Basic - File Handling · SD Basic - System and Environment · SD Basic - Program Control · SD Basic - Debugging.